

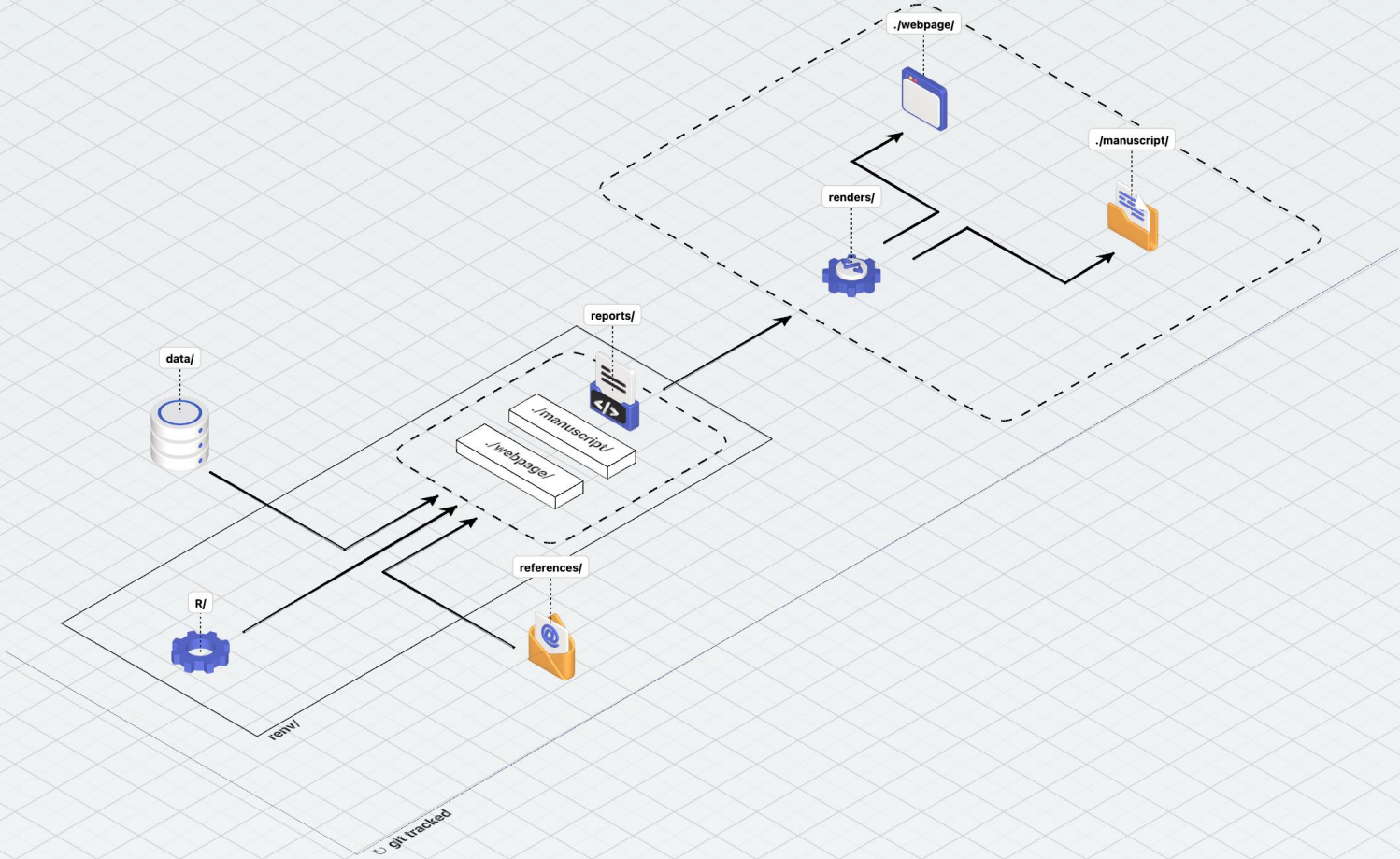
Consumer Decision-Making Analysis in R

Reproducible, Collaborative, and AI-Aware Workflows

Overview

Structure of the workshop

- ☒ Day 1: The scaffold
- ☒ Day 2: R and data analysis
- ☐ Day 3: Supercharge





user

Interfaces

Console (CLI)

`mean(mydata$age)`

Button (GUI)

Clicks: Analyze > Descriptive Statistics >
Descriptives > add variable(s) > ok

same

Instruction (expression)

`mean(mydata$age)`

R Engine / interpreter

1. lookup ``mydata`` in the workspace
2. lookup ``$age`` in ``mydata``
3. calculate mean of ``$age``

Output

`#> 41`

State (workspace)

`mydata`

The three-day roadmap

Day 1: Scaffold

- R
- Rstudio
- Git/GitHub basics
- Folders
- Data protection
- Quarto

Day 2: Analyse

- Base R
- Import
- Clean
- Transform
- Visualise
- Describe
- Model
- Moderate / mediate
- Collaborate

Day 3: Supercharge

- Basic LLM intro
- The power of context
- Ethical AI-use
- Local LLMs
- AI workflows
- Reproducibility check

Day 3

The Hypercharge

Efficient Ins and Outs

In six years, "AI" went from autocomplete to colleague

```
JS server.js > ...  
1  const express = require('express');  
2  const app = express();  
3  
4  var server = express();  
5  ✨  
6  server.  
   stack  
   subscribe  
   toString  
   trace  
   unlink  
   unlock  
   unsubscribe  
   use  
   abc app  
   abc express  
   abc require  
   abc server
```

2020

I finished solving all your problems!



2026

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
nshazeer@google.com

Niki Parmar*
Google Research
nikip@google.com

Jake Luke*
Google Research
jake@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show that models trained on the WMT 2014 English-to-French translation task improve over the existing best results, including those based on complex, recurrent or convolutional neural networks. On the WMT 2014 English-to-French translation task, our model achieves a new state-of-the-art (61.1 score of 41.0 after reranking) on a one-to-one model evaluation of the set of all 61.1 score of 41.0 after reranking for 15 days on eight GPUs, a small fraction of the training costs of the best models to the literature.

1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [29, 2, 5]. Transformer models have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

A major contribution of this paper is to show that proposed models (RNNs with self-attention and state-of-the-art) can be replaced by a simple, fast, and efficient architecture. The Transformer model and state-of-the-art have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

The effort to establish this idea. Additionally, the Transformer model and state-of-the-art have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

has been directly involved in every aspect of this work. Noam Shazeer would like to thank Google Brain for their support and the Transformer model and state-of-the-art have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

are: Niki Parmar, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. We thank Google Brain for their support and the Transformer model and state-of-the-art have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

efforts and the Transformer model and state-of-the-art have been proposed to push the boundaries of recurrent language models and machine translation [21, 21, 17].

Work performed while at Google Brain.

Work performed while at Google Brain.

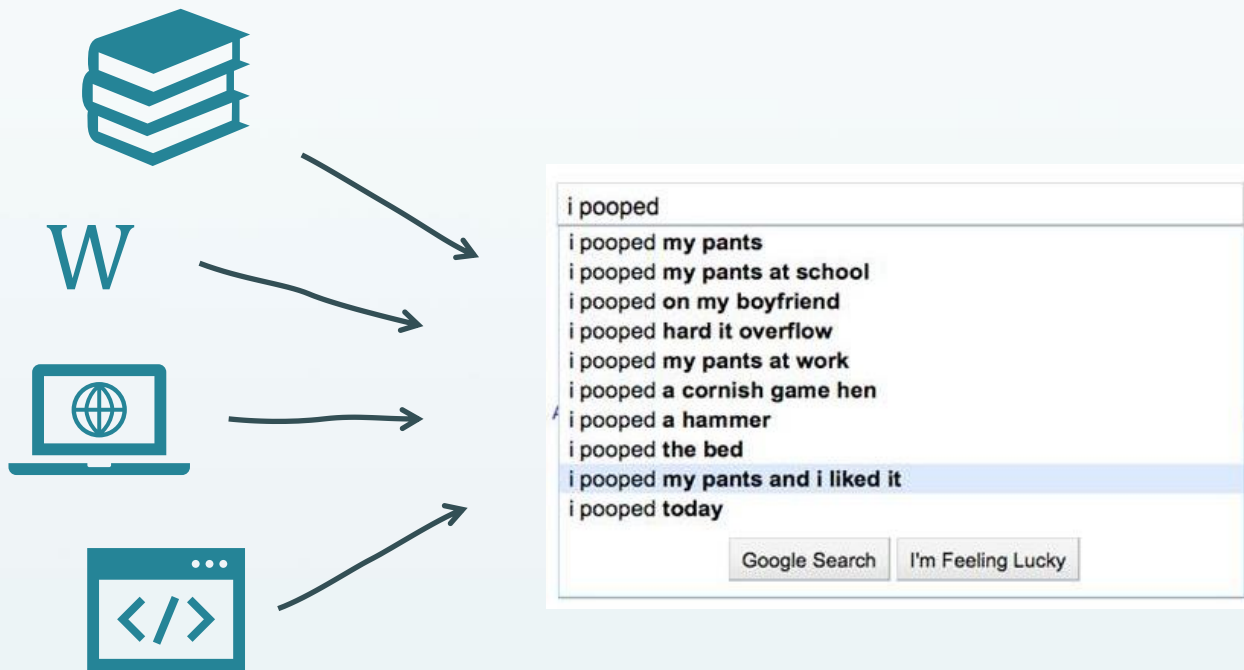
Vaswani et al. (2017)
Cited by 251,236

Directly connect every word to every other word.

Model relationships, not sequences.

Context is everything.

A Large Language Model is a next-word predictor



It does not "look things up." It predicts what comes next, one token at a time.

Exercise – Are you a LLM

Prompt: "The capital of France is ____"

Model: Paris (99% – it has seen this pattern endlessly)

Prompt: "library(tidyverse)\n consumer_data <- read_ ____"

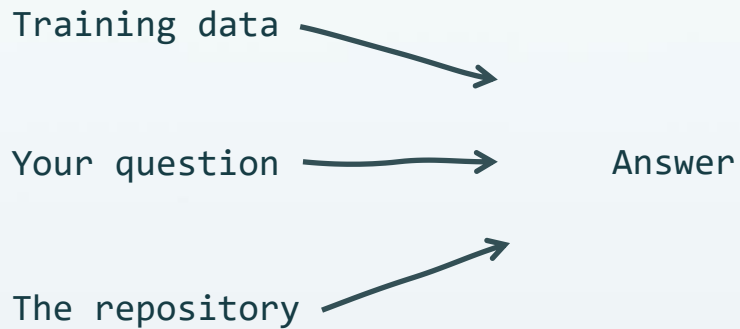
Model: csv((it learned how R code usually continues)

Prompt: "purchase_intention ~ condition + ____"

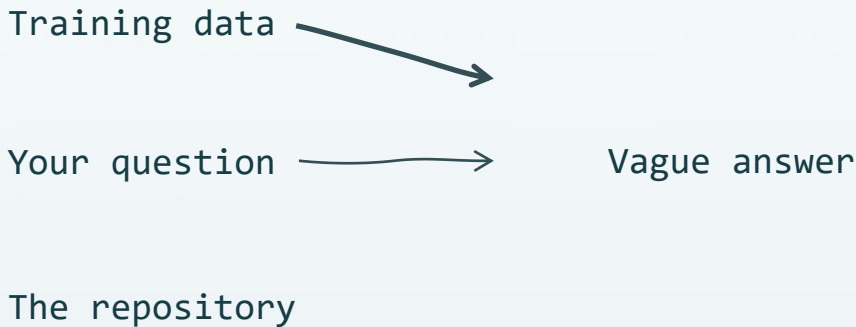
Model: price_sensitivity (a plausible predictor – but IS it in your data?)

1. "Given everything so far, what token is most likely next?"
2. "Given all other words in the text, which words are important?"

Strengths and traps, on one slide



Strengths and traps, on one slide



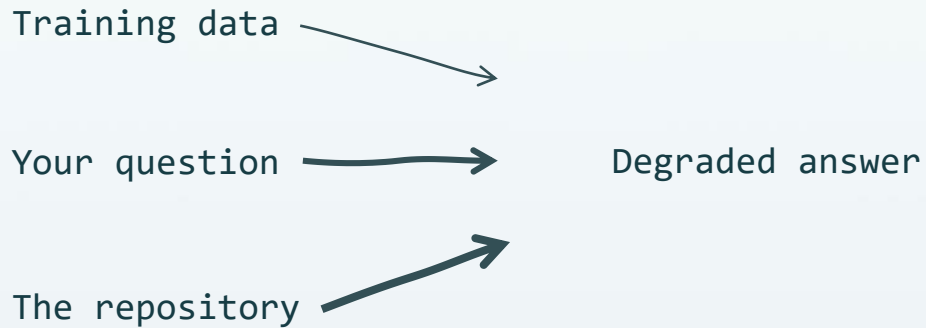
A vague prompt:

"write me a moderation analysis"
→ generic code, wrong variable
names, won't run

A repo-grounded prompt:

"following R/functions/analysis.R, add a
moderation of purchase_intention by
condition × price_sensitivity"

Strengths and traps, on one slide



Strengths and traps, on one slide



Precise instruction +

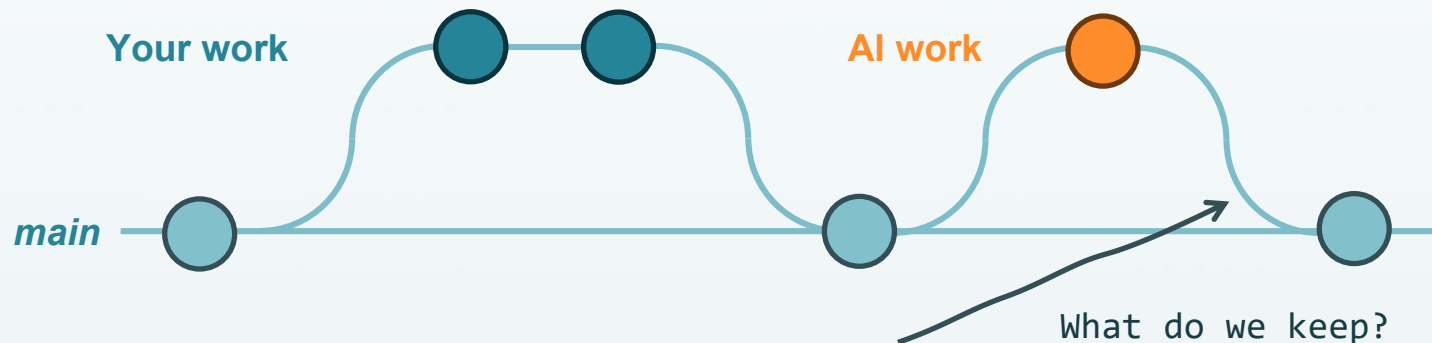
The repo supplies the missing context for free:

R/functions/	how we write functions
reports/*.qmd	how we report results
data/mock/	the real column names
README.md	what the project is

Everything that makes a repo reproducible also makes AI safe & useful

- 1. Open-source tools → the model has seen millions of examples like yours
- 2. Explicit structure → the repo IS the context the model needs
- 3. Git version control → you see exactly what the AI changed
- 4. Branches → let the AI try things without risking your work

Trust, but verify — the diff is your receipt



Changes from all commits • Jump to... • 318 -255

Unified	Split	Revert changes
13 post, invalid 14 -updates were a significant source of errors when updating owiCloud. 15 16 FAQ 17 --- 18 Why Did owiCloud Add Code Signing? 19 20 21 -By supporting Code Signing we add another layer of security by 22 ensuring that 23 nobody other than authorized person s can push updates for 24 applications and 25 ensuring proper upgrades. 26 27 Do We Lock Down owiCloud? 28 29 30 -The owiCloud project is open source and always will be. We do not 31 want to make 32 -it more difficult for our users to run owiCloud. Any code signing 33 errors on 34 -upgrades will not prevent owiCloud from running, but will display a 35 warning on 36 -the Admin page. For applications that are not tagged "Official" the 37 code signing 38 -process is optional. 39 40 Not Open Source Anywhere? 41 42	13 when updating owiCloud. 14 15 FAQ 16 --- 17 Why Did owiCloud Add Code Signing? 18 19 20 21 -By supporting Code Signing we add another layer of security which 22 ensures that 23 nobody other than authorized admins can push updates for 24 applications 25 this ensures proper upgrades. 26 27 Do We Lock Down owiCloud? 28 29 30 -The owiCloud project is open source and always will be. 31 32 We do not want to make it more difficult for our users to run 33 owiCloud. 34 -Any code signing errors on upgrades will not prevent owiCloud from 35 running, but will display a warning on the Admin page. 36 -For applications that are not tagged "Official" the code signing 37 process is optional. 38 39 Not Open Source Anywhere? 40 41	

A second editor — the one the AI tools live in



Visual Studio Code

<https://code.visualstudio.com> · Išversti šį puslapį

Visual Studio Code - The open source AI code editor | Your home for ...

Visual Studio Code is a free, open source AI code editor. Build with AI agents that plan, code, and debug for you. Manage multi-agent workflows across ...

Same project, different cockpit

RStudio

Files pane (bottom right)
Source / script editor
Console
Environment pane
Git tab (top right)
Packages / Plots / Help
Open .Rproj
Build ▶ Render

VS Code

Explorer (sidebar, top icon)
Editor (centre – same thing)
Terminal (View ▶ Terminal) + R console
(no built-in – use the R extension / Console)
Source Control (sidebar, branch icon)
Extensions add these (R extension, Quarto)
Open the FOLDER, or the .code-workspace file
Quarto extension: Render button / preview

Everything you know from RStudio exists in VS Code — just in a different panel.

Exercise — get VS Code running

1. Go to <https://code.visualstudio.com> → Download for your OS.
2. Install, open it, log in with GitHub
3. File ► Open Folder... → open your forked the-science-repository.
(or double-click the-science-repository.code-workspace)
4. Find these in the left sidebar:
 - Explorer (the file tree – your folders)
 - Source Control (the Git panel)
 - Extensions (the puzzle-piece icon – next slide)

CHECK

Can you see R/, reports/, and data/ in the Explorer?
Does the Source Control panel show your repo's branch?

WATCH OUT

RStudio is still much simpler to run R scripts and console commands. But your files update in real-time.

Turn VS Code into an R + Quarto + AI workbench

Open the Extensions panel (puzzle icon) and install:

R	(REditorSupport)	run R, syntax, data viewer, plots
Quarto	(Quarto)	render & preview .qmd, just like RStudio
Continue	(Continue.dev)	plug in a LOCAL or cloud model → next section
GitHub Copilot	(GitHub)	inline suggestions; has a FREE tier

Git/Source Control is already built in – nothing to install.

R + Quarto give you the workflow; Continue + Copilot add the AI.

The one rule that makes all of this safe

An AI assistant can only help with what it can SEE.
That's also the risk: confidential data must stay out of its view.

The recurring question (Day 3 version):

"What is safe to commit, share, upload to Overleaf, or paste into an AI tool?"

Scripts	usually yes
Raw data	no
Outputs	only if no one can be identified
Confidential data	never into external AI tools without approval

A rule to ring them all



Told your colleagues you did
all this by yourself

Deidentified participant data processed by a
commercial LLM

Sensitive, identifiable participant data processed by
a commercial LLM that uses your data for training

Your participants health data on Facebook
with full name

Open the repo with private data hidden

```
the-science-repository.code-workspace    → normal working view  
the-science-repository.llm.code-workspace → LLM-safe view ← use with AI
```

The .llm workspace hides the private folders from the editor entirely:

```
"files.exclude": { "data/raw": true, "data/processed": true }  
"search.exclude": { "data/raw": true, "data/processed": true }
```

Editor-integrated assistants (Continue, Copilot, Cursor) can't suggest, search, or read what the editor doesn't show. data/mock/ stays visible – safe to use.

The repo refuses to let the assistant read raw data

`.claude/settings.json` hard-blocks AI tools from your private data:

```
"permissions": {  
  "deny": [  
    "Read(./data/raw/**)",    "Read(./data/processed/**)",  
    "Glob(./data/raw/**)",    "Grep(./data/raw/**)",  
    "Bash(cat ./data/raw/*)", "Bash(head ./data/raw/*)"  
  ]  
}
```

Even if you ASK the assistant to open a raw file, it is refused.

Hiding (workspace) + blocking (.claude) = two independent seatbelts.

Hugging Face, LM Studio, Continue



Hugging Face

find open models



LM Studio

run your own models



Continue

use your models in VS Code

Local LLM (no internet connection needed)



Told your colleagues you did
all this by yourself

~~Deidentified participant data processed by a
commercial LLM~~

~~Sensitive, identifiable participant data processed by
a commercial LLM that uses your data for training~~

Your participants health data on Facebook
with full name

Start tiny — you can always go bigger

Old / modest laptop, just to see it work:

Qwen2.5-0.5B-Instruct	~0.4 GB	runs almost anywhere, even CPU-only
Llama-3.2-1B-Instruct	~1 GB	a small step up

A bit more RAM (≈ 8 GB free):

Llama-3.2-3B / Gemma-2-2B	~2-3 GB	noticeably more capable
---------------------------	---------	-------------------------

In LM Studio: pick a "GGUF" build, prefer a "Q4" quantisation (smaller, faster).

If even the tiny model is too slow $\rightarrow$ that's expected, and the point of the NEXT section: heavy lifting goes to a frontier model in the cloud.

Start tiny — you can always go bigger



Qwen2.5-0.5B-Instruct
0.4 GB Ram



Qwen3.6 35B
48 GB+ VRAM

Exercise — your first local "hello world"

1. Download LM Studio from <https://lmstudio.ai> and open it.
2. Search for Qwen2.5-0.5B-Instruct → download the Q4 GGUF build.
3. Load the model (top bar) and open the chat.
4. Type your hello-world prompt:

"What is this Repository about?"

5. Watch tokens appear one at a time – that's next-word prediction, live.

CHECK

Got a sentence back? You just ran an LLM entirely on your own machine. 🎉
Too slow or won't load? Pair up with a neighbour, or skip to frontier models –
the concept is what matters, not the horsepower.

When you want real power

Claude (Anthropic) – strong at code, reading large projects, careful reasoning
Codex / ChatGPT (OpenAI) – strong all-rounders, big ecosystem

These can:

- read your entire repo for context
- plan a change, edit several files, run code, and fix its own errors
- work from a plain-English description of what you want

The skill that unlocks them is not coding. It's PROMPTING + CONTEXT.

One prompt, a whole analysis — done to the repo's standards

```
"Read R/functions/analysis.R and reports/webpage/03-analysis.qmd to learn this project's conventions. Then, on a new branch:
```

1. Add a function to R/functions/analysis.R that runs a moderation of `purchase_intention ~ condition * price_sensitivity`. Improve style, output, and run assumption tests.
2. Add a matching interaction plot to R/functions/plotting.R using our theme.
3. Add a section to reports/webpage/03-analysis.qmd that calls both, shows the figure, and writes a 3-sentence results paragraph with cautious wording.

```
Explain to me what you changed."
```

Context · Task · Constraints · Review

CONTEXT	point at real files: "follow R/functions/analysis.R"
TASK	be specific and numbered: what, exactly, in what order
CONSTRAINTS	set the guardrails: "work on a branch", "match our style", "use tidyverse"
REVIEW	"show me the diff / explain before you change files"

Weak: "write me a moderation analysis"

Strong: all four boxes above, filled in.

*A good prompt is a good task brief. Imagine talking to your research assistant.
Always be nice! (not actually important but good for karma)*

For multi-step work, write it down first

For anything bigger than one prompt, don't free-style – structure it:

1. PRD ask the model to write a short plan first:
 prd.md – goal, scope, what "done" looks like, what NOT to do.
2. TASKS break the PRD into a checklist (a task graph):
 tasks.json – small, checkable steps, in order.
3. LOOP feed it the plan + checklist and let it work through, one task
 at a time, ticking each off – re-run it on the list until the
 boxes are all green. (the "Wiggum loop": simple, persistent, repeat.)

*Plan in plain text, in the repo. The plan is context AND a record of decisions.
Be open to new developments!*

Exercise — your turn to drive

1. In VS Code, sign in to GitHub Copilot (free tier – no card needed).
2. Open YOUR CODING AGENT Chat (Claude Code, Codex, GH Copilot, Continue, ...) in this repo (the .llm workspace).
3. Right-click R/functions/plotting.R ▶ Copy Relative Path.
4. Write a Context · Task · Constraints · Review prompt, e.g.:

“`Read R/functions/plotting.R`. Add a function that plots the distribution of purchase_intention by condition, matching the existing ggplot style. Show me the code and explain it before changing any file. Don't touch data/.”

5. Read what it gives you. Would it run? Is it in our style? Improve the prompt.

CHECK

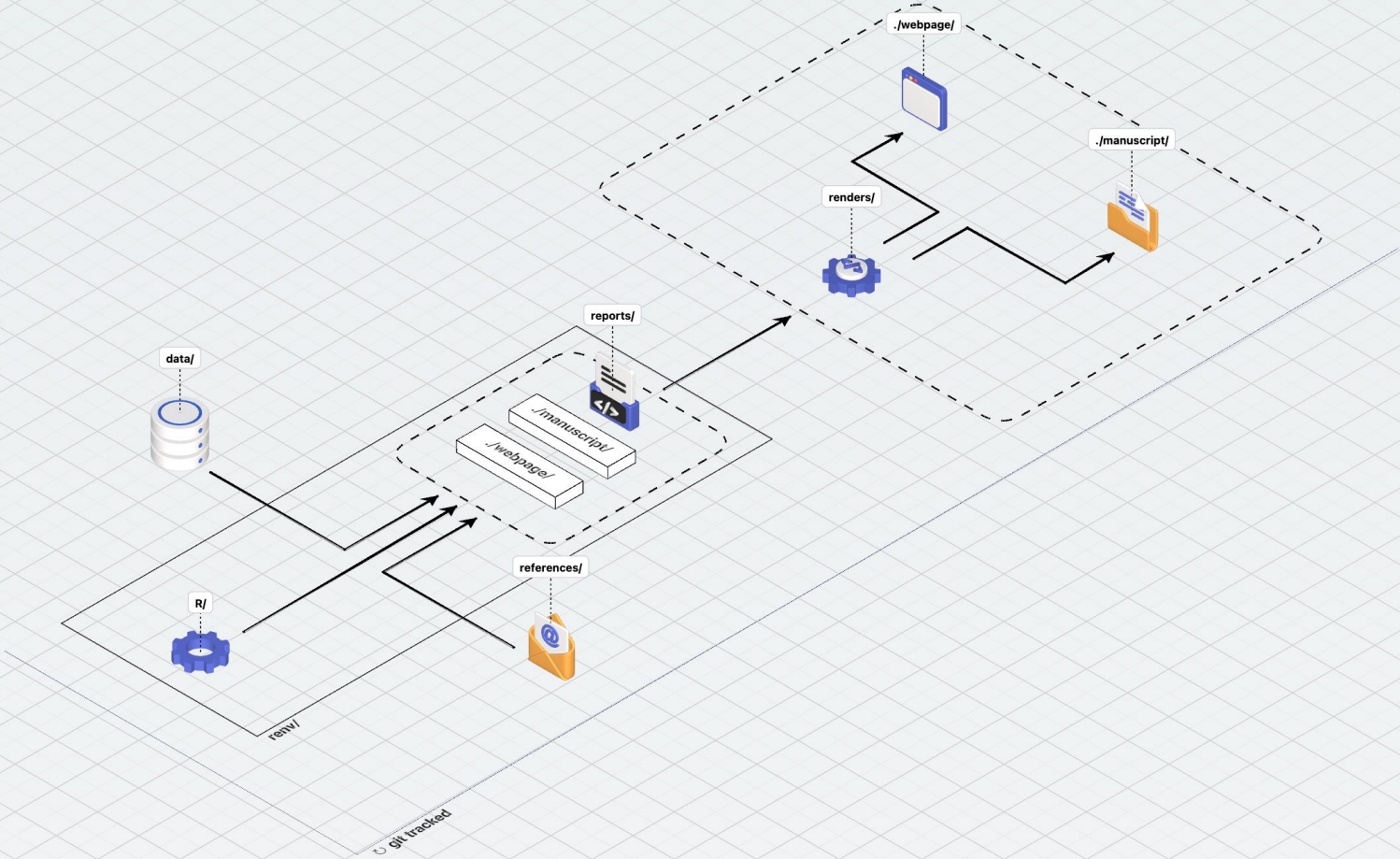
Did adding the file path change the quality of the answer?

Can YOU tell whether the suggestion is correct? (that's the skill that matters)

Recap

A journey

What we learned in three
days





user

Interfaces

Console (CLI)

`mean(mydata$age)`

Button (GUI)

Clicks: Analyze > Descriptive Statistics >
Descriptives > add variable(s) > ok

same

Instruction (expression)

`mean(mydata$age)`

R Engine / interpreter

1. lookup ``mydata`` in the workspace
2. lookup ``$age`` in ``mydata``
3. calculate mean of ``$age``

Output

`#> 41`

State (workspace)

`mydata`

In six years, "AI" went from autocomplete to colleague

```
JS server.js > ...
1  const express = require('express');
2  const app = express();
3
4  var server = express();
5  ✨
6  server.
  stack
  subscribe
  toString
  trace
  unlink
  unlock
  unsubscribe
  use
  abc app
  abc express
  abc require
  abc server
```

2020

I finished solving all your problems!



2026

- [✓] GitHub account created
- [✓] Repository forked to your account
- [✓] GitHub Pages on → your website loads
- [✓] First commit made (README link points to your site)
- [✓] R and RStudio installed
- [✓] Fork cloned and opened via the .Rproj
- [✓] `renv::restore()` finished successfully
- [✓] quarto render reports/webpage works locally

- [✓] Base R
- [✓] R scripts
- [✓] R functions
- [✓] Quarto reports (HTML, Word, LaTeX)
- [✓] Multi-step R workflow

- [✓] LLM foundations
- [✓] LLMs and VS Code
- [✓] Local LLMs
- [✓] ethical LLM use
- [✓] AI workflows

Thanks for the fun time

Questions or projects, just contact us :)

info@thedataflowcompany.com
www.thedataflowcompany.com